

Session 4 · Deck 4.A3

Optimization in Many Dimensions

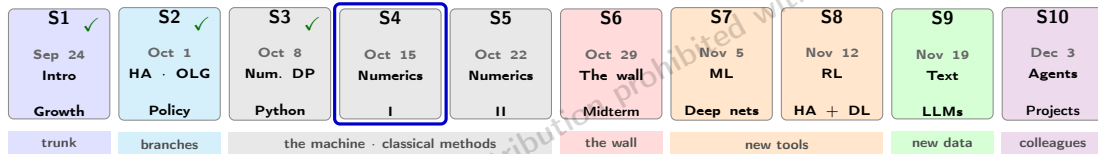
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.A3 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 4.A1 root-finding · 4.A2 optimization on an interval · **4.A3** optimization in many dimensions · 4.A4 differentiation.

What Breaks When the Choice Is a Vector

Every method in 4.A2 worked by **shrinking a bracket**.

An interval has two ends and a clear inside. A region of $\mathbb{R}^N$ has neither.

What replaces it is a **local quadratic model**:

$$f(\theta + \Delta) \approx f(\theta) + g^\top \Delta + \frac{1}{2} \Delta^\top H \Delta,$$

with $g = \nabla f$ and $H = \nabla^2 f$. Setting its gradient to zero gives the Newton step

$$\Delta = -H^{-1}g.$$

Cost table adapted from QM. *Numerical Methods*, "Why Multivariate is Hard".

N	Hessian entries	flops to solve
10	100	333
100	10,000	333,333
1,000	1 million	333 million

And that is *per step*. Storing H is $O(N^2)$, factorising it $O(N^3)$, and building it by finite differences costs $O(N^2)$ function evaluations on top.

Quasi-Newton: Learn the Curvature From the Steps You Took

You already know something about H after one step. If you moved by $s = \theta_1 - \theta_0$ and the gradient changed by $y = g_1 - g_0$, then for a quadratic

$$H s = y.$$

That is the **secant condition**. Quasi-Newton methods carry an approximation B and update it so that $B_1 s_0 = y_0$ holds exactly, while changing B as little as possible.

BFGS is the rank-two update that does this and keeps B positive definite:

$$B_1 = B_0 + \frac{y_0 y_0^\top}{y_0^\top s_0} - \frac{B_0 s_0 s_0^\top B_0}{s_0^\top B_0 s_0}.$$

What it buys

No second derivatives, ever. The per-step cost falls from $O(N^3)$ to $O(N^2)$, and L-BFGS — which stores only the last few (s, y) pairs — brings it to $O(N)$.

This is the same trade as Broyden in 4.A1: pay a few more iterations, stop building derivative matrices.

BFGS, Worked Through by Hand

Minimize $f(x, y) = \frac{1}{2}x^2 + y^2$ from $\theta_0 = (2, 2)$, so $g_0 = (2, 4)$ and $H = \text{diag}(1, 2)$.

1. $B_0 = I$, so $d_0 = -g_0 = (-2, -4)$.
2. Step $\alpha_0 = 0.5$: $\theta_1 = (1, 0)$, $g_1 = (1, 0)$.
3. $s_0 = (-1, -2)$, $y_0 = (-1, -4)$.
4. $y_0^\top s_0 = 9$, $s_0^\top B_0 s_0 = 5$.
5. $B_1 = \begin{pmatrix} 41/45 & 2/45 \\ 2/45 & 89/45 \end{pmatrix} = \begin{pmatrix} 0.911 & 0.044 \\ 0.044 & 1.978 \end{pmatrix}$.

Check the secant condition

$B_1 s_0 - y_0$ has largest entry 4×10^{-16} — it holds **exactly**, not approximately.

Sources that print B_1 rounded to two decimals and then report “ $B_1 s_0 \approx y_0$ ” are describing their own rounding, not the method. The condition is imposed by construction.

Sanity check: the true Hessian gives $H s_0 = (-1, -4) = y_0$. One step has already taught B the curvature along s_0 .

Adapted from QM *Numerical Methods*, BFGS example. Note: the source's $\alpha_0 = 0.5$ is a convenient number, not the exact line-search minimizer, which is $5/9$.

The Step Size: What a Line Search Is For

The update gives a *direction* d ; how far to move along it is a separate, one-dimensional problem:

$$\varphi(\alpha) = f(\theta + \alpha d), \quad \alpha > 0.$$

Solving it exactly is wasteful — the direction will change next step anyway. What is needed is a step that makes **enough** progress:

- **Armijo**: f falls by a fixed fraction of what the slope promised.
- **Curvature**: the slope has flattened enough that we are not stopping too early.

Together these are the **Wolfe conditions**; backtracking from $\alpha = 1$ is the usual implementation.

Adapted from QM *Numerical Methods*, "Interlude: What is a Line Search?".

This is 4.A2, nested

The inner problem is exactly the scalar maximization of the previous deck — which is why golden section and Brent turn up inside multivariate optimizers.

The curvature condition is not optional for BFGS: it is what guarantees $y^T s > 0$, and hence that B stays positive definite. Skip it and the update can produce a direction that is not even uphill.

Conjugate Gradient: N Steps, and No Matrix at All

Steepest descent zig-zags: each step undoes part of the last, because consecutive directions are orthogonal rather than *complementary*.

Conjugate gradient fixes the direction instead of the step. Choose d_n **H -conjugate** to everything already used,

$$d_i^\top H d_j = 0 \quad (i \neq j),$$

by the Fletcher–Reeves recursion

$$d_{n+1} = -g_{n+1} + \frac{g_{n+1}^\top g_{n+1}}{g_n^\top g_n} d_n.$$

Each step then removes one direction's error **permanently**.

Exact in N steps

On our quadratic, from $(2, 2)$:

$$(2, 2) \rightarrow (0.889, -0.222) \rightarrow (0, 0),$$

converged to machine zero in **two** steps in two dimensions — as the theory promises.

And H never appears. CG needs only the ability to *multiply* by H — or, in the non-linear version, nothing but gradients.

Adapted from QM-Lec. 4's conjugate-gradient block. Note: the source's printed $d_1 = [-80/81, 40/81]$ has second component $20/81$; see the closing frame.

The Memory Wall

The reason CG exists is not elegance. It is that B does not fit.

N	Newton / BFGS	L-BFGS	CG
10^2	10^4	2,000	300
10^4	10^8	2×10^5	3×10^4
10^6	10^{12}	2×10^7	3×10^6

At $N = 10^6$ the dense approximation is 10^{12} doubles — **8 terabytes**. L-BFGS needs 160 MB and CG needs 24.

L-BFGS

Keep the last $m \approx 10$ pairs (s, y) and rebuild the action of B^{-1} on demand. $O(mN)$ memory, and almost all of BFGS's behaviour.

$N = 10^6$ is not hypothetical. It is the parameter count of a small neural network — which is why **no one trains one with Newton's method**, and why Session 7's optimizer is a first-order one.

When the Objective Is a Sum Over Data

A different kind of large problem. Suppose

$$L(\theta) = \frac{1}{M} \sum_{i=1}^M \ell(\theta; z_i),$$

with M in the millions — an empirical risk, a simulated method of moments objective, a likelihood over a long panel.

Here the trouble is not the dimension of θ but the **cost of one gradient**: every step reads all M observations, so a single BFGS iteration is a full pass over the data.

Stochastic gradient descent replaces the full gradient with the gradient of a random subset:

$$\theta_{n+1} = \theta_n - \alpha_n \nabla_{\theta} \ell(\theta_n; z_{i_n}).$$

Adapted from QM Lec. 4's SGD block.

Why it is not madness

The sampled gradient is **unbiased**: its expectation over the draw is the full gradient. You are taking a noisy step in a direction that is right on average.

The trade is stark — M times cheaper per step, and a step that is wrong. Whether that is a good bargain depends entirely on how much the observations agree with each other.

The Noisy Path, and the Dimension Penalty

Split our $f = \frac{1}{2}x^2 + y^2$ into $\ell_1 = \frac{1}{2}x^2$ and $\ell_2 = y^2$, and draw one at a time with $\alpha = 0.5$:

θ_n	drawn	θ_{n+1}
(2, 2)	ℓ_2	(2, 0)
(2, 0)	ℓ_1	(1, 0)
(1, 0)	ℓ_1	(0.5, 0)
(0.5, 0)	ℓ_2	(0.5, 0)
(0.5, 0)	ℓ_1	(0.25, 0)

Row four makes no progress at all — the draw carried no information about x . That is the noise, and it is the price of the cheap step.

Trajectory from QM Lec. 4's SGD example, recomputed. Note: the source defines f as the *sum* and then differentiates it with $1/M$ averaging, giving

$\nabla f(2, 2) = [1, 2]$ — which contradicts the $[2, 4]$ its BFGS and CG examples use for the same function, and contradicts this table. The consistent definition is used here.

Why α must shrink with N

Noise of unit size in each coordinate has norm $\sqrt{N}$:

N	$\ \text{noise}\ $
1	1
10^4	100
10^6	1,000

so a step safe in one dimension is a thousand times too large in a million. Convergence needs $\sum \alpha_n = \infty$ and $\sum \alpha_n^2 < \infty$ — Robbins–Monro.

When You Have No Derivatives at All

Nelder–Mead keeps a simplex of $N + 1$ points and moves it by reflecting the worst vertex through the centroid, then expanding, contracting or shrinking. No derivatives, no model of f , and no convergence theory worth the name — but it is robust to noise and kinks.

Rosenbrock from $(-1.2, 1)$	evals	$ \theta - \theta^* $
BFGS	117	9×10^{-6}
Nelder–Mead	● 249	5×10^{-11}
Powell	607	2×10^{-13}

Read that carefully

BFGS is fastest by a factor of two, and *least* accurate by five orders of magnitude — it stops when its gradient test is satisfied, which on a curved valley happens early.

“Fastest” and “most accurate” are different questions. Decide which one you are asking **before** you pick the method, and report the tolerance you used.

Simulated Annealing: Accept a Worse Point, On Purpose

Nelder–Mead needs no derivatives but still walks downhill, so it still stops at the first optimum it meets. When the objective is **multimodal**, that is the wrong answer, and no amount of accuracy fixes it.

Simulated annealing proposes a random move and accepts it with probability

$$\min \left\{ 1, \exp(\Delta f / T_n) \right\},$$

so an *uphill* move is always taken and a downhill one is sometimes taken — often when T is high, rarely as $T_n \rightarrow 0$. Cooling slowly enough ($T_n \sim c / \ln n$) gives convergence to the global optimum in probability.

Adapted from QM-Lec. 4, “Simulated Annealing: MCMC for Optimization”.

from $x_0 = 8$	x	$f(x)$
local method	8.35	-0.50
annealing	0.00	3.00
truth	0.00	3.00

On $-x^2/20 + 3 \cos 3x$ the local method is trapped in the valley it started in. Annealing walks out of it and finds the global maximum — in 20,000 evaluations, which is the price.

One Accept–Reject Rule, Two Very Different Uses

The rule on the last slide is not really an optimization idea. It is the **Metropolis–Hastings** rule, and the only thing that changes between its two uses is what you do with the temperature.

	annealing	Metropolis–Hastings
accept w.p.	$\min\{1, e^{\Delta f/T_n}\}$	$\min\{1, \frac{\pi(\theta^*)}{\pi(\theta_n)}\}$
temperature	driven to 0	held at 1
you want	the argmax	the distribution
output	one point	a whole chain

Why an economist cares

Hold the temperature at one and the chain's stationary distribution *is* the posterior. That is how a structural model is estimated when the likelihood can be evaluated but not maximised reliably.

We use the optimization half today. The sampling half returns in **Session 9**, where calibration and estimation are the bridge from models to data — and where the object being sampled is a posterior over parameters rather than a maximum.

Comparison adapted from QM Lec. 4, “RWMH vs. Simulated Annealing” and “When to Use?”.

When Constraints Bind

Almost every economic optimization is constrained:

$c > 0$, $n \in [0, 1]$, $a' \geq -\underline{a}$. At an optimum the Karush–Kuhn–Tucker conditions hold: $\nabla_x \mathcal{L} = 0$, $\mu \geq 0$, $h(x) \geq 0$, and

$$\mu h(x) = 0.$$

It is the last one — **complementary slackness** — that is awkward numerically: it is not an equation you can hand to a solver.

The Fischer–Burmeister function turns it into one:

$$\phi(a, b) = a + b - \sqrt{a^2 + b^2},$$

$$\phi(a, b) = 0 \iff a, b \geq 0, ab = 0.$$

$\phi(a, b) = a + b - \sqrt{a^2 + b^2}$ in the instructor's own notation, from QM Lec. 8, "Handling Inequality Constraints".

Where you met this

2.A1 used exactly this ϕ to write the borrowing-constrained household's optimality conditions as a square system — and pointed here for how to solve it.

With ϕ in hand the KKT system becomes $F(x, \mu) = 0$, and 4.A1's machinery applies directly: Newton, a Jacobian, a line search.

ϕ is not differentiable at the origin — exactly where the constraint is marginally binding — so solvers smooth it and drive the smoothing to zero.

Choosing, and a Warning About Worked Examples

Method	Use it when
Newton	N small, H cheap and reliable
BFGS	the default for smooth f up to a few hundred parameters
L-BFGS	thousands of parameters; only $O(N)$ memory
Nelder–Mead	f noisy, kinked, or a black box

Check the arithmetic in your sources

Recomputing the conjugate-gradient example that accompanies this material turned up a printed direction $d_1 = [-80/81, \mathbf{40/81}]$ whose second component should be **20/81** — and with the printed value the next two lines of the derivation do not follow.

Worked examples are the most useful thing in a numerical-methods course and the easiest place for an error to survive, because nobody re-runs arithmetic that is already typeset. **Re-run it.**

Takeaways

- In $\mathbb{R}^N$ there is no bracket to shrink. A **local quadratic model** replaces it, and the Hessian is what makes that expensive: a million entries and 333 million flops per step at $N = 1,000$.
- The **secant condition** $Hs = y$ is free information from the step you just took. BFGS imposes it exactly — verified to 4×10^{-16} , not “approximately”.
- Direction and step length are separate problems; the step length is **4.A2, nested**.
- **Fast and accurate are different questions**. On Rosenbrock, BFGS used half the evaluations of Nelder–Mead and finished five orders of magnitude further away.
- Binding constraints are handled by turning complementary slackness into an equation — Fischer–Burmeister — after which everything reduces to **4.A1**.
- When N is enormous the Hessian stops fitting — 10^{12} entries at $N = 10^6$ — so **CG** and **L-BFGS** replace it; when the objective is a sum over millions of observations, **SGD** replaces the gradient. Both return in Session 7.
- When the objective is **multimodal**, every method here stops at the first optimum it meets. Annealing escapes by accepting worse points on purpose — and the same accept–reject rule, at fixed temperature, is how you *sample* a posterior (Session 9).

Bridge: All of This Assumed You Had Derivatives

Newton needs a gradient and a Hessian. BFGS needs a gradient. The Jacobian in 4.A1 needs N^2 partial derivatives. Every method in the last two decks has quietly assumed that ∇f arrives from somewhere.

For a model you wrote down, it does — you can differentiate by hand, once, and check it. For a model with a hundred equations, or a solver nested inside another solver, hand-differentiation stops being realistic. And the obvious substitute, a finite difference, has a failure mode we have already measured.

Next (4.A4)

Where derivatives come from: forward and central differences, the step size that is optimal and why, the complex-step trick that has no cancellation at all, and automatic differentiation.

Then Newton, run three ways, with the Jacobian supplied by each.